SKJS verifier · research note

Typing Promise Resolution

Native promise contracts and a sound generic Awaited<*T*>, and where TypeScript's own is unsound

Cristiano Calcagno · 9 October 2026

Outline

1. The problem: generic forwarding has no sound type
2. TypeScript's Awaited and why it is unsound
3. Native promise contracts
4. Generic Awaited<*T*>: implied bounds
5. Results: target programs, census, remaining limits

1. Generic forwarding has no sound type

```
async function wrap<T>(x: T): Promise<Awaited<T>> { return x; }
```

The trusted contracts on the base branch:

$$\%\text{Await}\% : \quad \forall T, U \le \text{Prim}.\ (\text{Promise}\langle T\rangle \mid U) \Rightarrow T \mid U$$

$$\%\text{AsyncReturn}\% : \quad \forall T, U \le \text{Prim}.\ (T \mid \text{Promise}\langle U\rangle) \Rightarrow \text{Promise}\langle T \mid U\rangle$$

T may be a promise or an object, and no type says what x becomes once a promise resolves with it. So wrap is refused.

Definitions

Prim: the primitives. Obj: every other value.

Thenable: an object with a callable then.

Resolution: a primitive or non-thenable fulfills the promise; a thenable's then is called.

Promise<*B*>: a native promise whose fulfillment values lie in *B*.

N: values that can never have a callable then.

2. TypeScript's Awaited

```ts
type Awaited<T> =
  T extends null | undefined ? T : // special case
  T extends object & { then(onfulfilled: infer F, ...args: infer _): any } ?
    F extends ((value: infer V, ...args: infer _) => any) ?
      Awaited<V> : // unwrap recursively
    never :
  T; // no declared then
```

- Decided from the *declared* type; distributes over unions; deferred on a type variable.
- Used by Promise.resolve, Promise.all, and to type await x.

Awaited is unsound on object types

Example 1 (object types are open). Accepted by tsc --strict; Node gives "hi".

```
function asRecord(x: { n: number }): { n: number } { return x; }
const t = { n: 1, then(resolve: (v: string) => void) { resolve("hi"); } };
const v = await asRecord(t); // TS: { n: number } Node: "hi"
```

Example 2 (resolution is not idempotent). Accepted by tsc --strict; Node gives 42.

```
const obj = { value: 1 };
const p: Promise<{ value: number }> = Promise.resolve(obj);
Object.assign(obj, { then: (r: (x: number) => void) => r(42) });
const first = await p;     // TS: { value: number } Node: obj
const second = await first; // TS: { value: number } Node: 42
```

Resolution can happen twice

```
const o = { n: 1 };  const p = Promise.resolve(o);  Object.assign(o, { then: (r) => r(42) });
```

Operation on p	Fulfills with
`await p`	the object o
`Promise.resolve(p)`	the object o (it returns p itself)
`Promise.all([p]) element`	the object o
`async () => p`	42
`p.then(), p.then(x => x)`	42
`p.catch(…), p.finally(…)`	42
`new Promise(r => r(p))`	42

Observing a fulfillment value is safe at any payload; resolving it again is safe only if it stays non-thenable.

3. Native promise contracts

Promise<*B*> is a native promise of the realm, with stable standard prototype, constructor, methods and species. A result that is resolved again must stay non-thenable for its whole lifetime:

> $N = \text{Prim} \mid \{\ \text{readonly then?: Prim}\ \} \mid \text{readonly unknown[]}$
>
> $\%\text{Await}\% : \forall T, U \le N.\ (\text{Promise}\langle T\rangle \mid U) \Rightarrow T \mid U$
>
> $\%\text{AsyncReturn}\% : \forall T \le N, U \le N.\ (T \mid \text{Promise}\langle U\rangle) \Rightarrow \text{Promise}\langle T \mid U\rangle$
>
> $\%\text{PromiseResolve}\% : \forall T, U \le N.\ (\text{Promise}\langle T\rangle \mid U) \Rightarrow \text{Promise}\langle T \mid U\rangle$

await admits any promise payload *T*, and a value in *N* at its own type; async return, Promise.resolve of a plain value, catch and finally require *N*. These are the instances, at concrete types, of the qualified intrinsics of slide 9.

4. Implied bounds: Awaited<*T*> constrains *T*

```
async function wrap<T>(v: T): Promise<Awaited<T>>
async function log<T extends Awaitable>(v: T): Promise<void>
```

> wrap : ∀*T* : Resolvable. (v: *T*) ⇒ Promise<*T*.Result> log : ∀*T* : Awaitable. (v: *T*) ⇒ Promise<void>
> %Await% : ∀*X* : Awaitable. (*X*) ⇒ *X*.Observed %AsyncReturn% : ∀*X* : Resolvable. (*X*) ⇒ Promise<*X*.Result>

- ***T* : Awaitable**, written *T* extends Awaitable: *T* has an observation *T*.Observed, what await needs.
- ***T* : Resolvable**, written *T* extends Resolvable: *T* also has a result *T*.Result, what a fresh resolution yields, never thenable; async return needs it. Awaited<*T*> in a signature implies it, and denotes *T*.Result.
- **Not a subtype bound.** The values of *T* are unconstrained; each call supplies the classification of its type argument. Bounds are read off the signature.
- The other implied bound: a Promise argument of an imported or library signature implies $T \leq N$.

Who fixes T.Observed and T.Result

Rule	type argument X	X.Observed (Awaitable)	X.Result (Resolvable)
stable	$X \le N$	X	X
native	Promise<A>	A, any payload	A.Result
union	$A \mid B$	A.Observed | B.Observed	A.Result | B.Result

Exactness (a lemma): if $X \le N$, then X.Observed = X.Result = X. The three rules are the only ones, so a stable type has exactly one observation and one result.

Who decides. The OCaml producer proposes types only. Lean's proof search finds the classification of each call's type argument, and the Lean checker checks it. Inside a body the two types stay abstract: T.Result ≤ N, and T.Observed resolves to T.Result.

Awaited compared with *T*.Result

Input type	Awaited	*T*.Result
primitive, null, undefined	itself	itself (stable)
Promise<number>	number	number (native, stable)
Promise<Promise<number>>	number	number (native twice)
number | Promise<readonly string[]>	number | readonly string[]	number | readonly string[] (union)
{ n: number }	{ n: number }	no classification: refused
PromiseLike<number>	number	no classification: refused
type variable *T*	deferred	*T* : Resolvable, classified at each call

T.Result agrees with Awaited wherever the result stays non-thenable, and refuses where Awaited is unsound.

5. Target programs

Program	Status
`accept<T>(v: Awaited<T>), a cell of Awaited<T>`	verifies
`async function wrap<T>(v: T): Promise<Awaited<T>> { return await v; }`	verifies, at number, arrays, native promises, unions
`settle<T extends Resolvable>(v: T): Promise<Awaited<T>>`	verifies
`log<T extends Awaitable>(v: T), at Promise<{ n: number }>`	verifies
`function back<T extends number>(v: Awaited<T>): T`	verifies (exactness)
`wrap<{ n: number }>(v) // open record`	refused
`erase (v: T) => Promise<Awaited<T>> to Promise<unknown>`	refused
`async (v: Promise<{ n: number }>) => v // the 42 case`	refused

All 903 pipeline cases meet their expected outcome; 5 generic Promise cases are recorded gaps.

Census of the SKJS corpus

SKJS corpus, units	30 September	9 October
accepted by SKJS	2,674	2,674
verified by the verifier	2,466	2,467

No unit moved because of this branch: per-file reruns at its base give the same outcome for every unit that moved.

The six promise units	9 October
queryMixedAwaitedLiteralBroadLength: await of an array	verifies (await of a stable value)
testAwaitThenable, testAwaitThenableNested: await of a PromiseLike	refused on principle: thenable adoption
createCounterService, freshReturnSnapshot: async return of a fresh object	refused on principle: thenable adoption
testKeyof: keyof Promise<number>	gap: a unique symbol key

Remaining limits

- **Reactions:** then, catch and finally keep *N*-bounded contracts: a reaction's payload must be stable.
- **Thenable adoption** is refused on principle: await of a PromiseLike, and async return of a fresh object, would call a then the type does not describe.
- **Bounds naming another binder**, such as *T* extends *U*, get no stable-bound shortcut: Awaited<*T*> stays apart from *T*.
- **Higher-order uses** need generic function types, which are outside the fragment.
- **Further rules** would cover: values a Promise executor resolves, a second reaction on an already-settled promise, thenables that only resolve to themselves, and promises that can never fulfill.
- **Listed gaps:** explicit catch<string>, source Promise copies.